

第 37 讲

Momentum 动量

为什么优化器要“记住”过去的 Gradient

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

第 33–36 讲已经建立了普通 Gradient Descent / Mini-Batch SGD 的完整闭环：当前 Batch 给出 Gradient, Optimizer 根据 Learning Rate 更新参数。但是 Mini-Batch Gradient 会抖动，狭长 Loss Landscape 中不同方向的梯度还可能反复变号，导致参数在谷底两边来回震荡。本讲引入 **Momentum (动量)**：不再只看当前一步的 Gradient，而是保留一部分过去的更新趋势。它是理解 Adam、AdamW 的关键台阶。

学习目标

学完本讲，你应该能够：

1. 用通俗语言解释为什么普通 SGD 的“只看当前 Gradient”可能导致震荡；
2. 逐项解释 Momentum 更新式中的参数、Gradient、momentum buffer、系数 μ 和 Learning Rate η ；
3. 手算若干步 Momentum，并解释为什么持续同方向会被强化、正负交替会被部分抵消；
4. 看懂狭长 Loss Landscape 中 SGD 与 Momentum 的参数轨迹差异；
5. 理解 μ 越大不代表越好，必须与 LR、曲率、Gradient Noise 一起讨论；
6. 区分“Gradient 平滑”“参数惯性”“更快收敛”三个相关但不完全相同的说法；
7. 看懂 `torch.optim.SGD(momentum=0.9)` 在数学上做了什么；
8. 知道 PyTorch Momentum 与教材中其他 Momentum 公式为何形式不同但思想一致；
9. 理解 Momentum 与 Adam 的 first moment m_t 的关系；
10. 在 LLM / Agent Systems 论文中判断 momentum、 β_1 、LR、Batch Size 等超参数是否需要联合解读。

1 为什么普通 SGD 可能“走得很笨”？

最基本的 SGD 更新是：

$$\theta_{t+1} = \theta_t - \eta g_t$$

其中 g_t 是当前 Step 的 Gradient。这个规则有一个非常重要的特点：

它几乎只相信“现在这一刻”的 Gradient。

如果每一步 Gradient 都非常稳定，这没有问题。但真实神经网络训练通常存在：

- Mini-Batch sampling 带来的 Gradient Noise；

- Loss Landscape 不同方向曲率差异巨大；
- 某些方向长期都在下降，而另一些方向频繁正负翻转；
- 参数接近狭长山谷后，Gradient 可能不断把参数从谷底左边推到右边，再从右边推回左边。

原理

如果某个方向的 Gradient 长期保持同号，说明这个方向可能存在稳定的下降趋势；如果某个方向的 Gradient 正负频繁交替，说明参数很可能正在该方向上来回震荡。普通 SGD 对两种方向没有“历史记忆”，Momentum 就是为此引入历史信息。

2 先用一个生活比喻理解 Momentum

想象一个小球沿着山谷往低处滚。

普通 SGD 更像：

每走一步都停下来，只看脚下这一刻的坡度，然后重新决定下一步。

Momentum 更像：

除了看当前坡度，还保留之前已经形成的运动趋势。如果连续很多步都朝同一个方向下坡，这个方向上的“速度”会积累；如果坡度不断左右翻转，过去的惯性会与当前方向互相抵消一部分。

这个比喻并不是说神经网络里真的存在质量、速度和牛顿力学，而是帮助理解：**Momentum 给优化器增加了一个跨 Step 的状态变量。**

普通 SGD 每一步只需要：

$$\theta_t, \quad g_t$$

Momentum 还需要保存：

$$v_{t-1}$$

也就是说，Optimizer 开始“有记忆”了。

3 本课程采用的 Momentum 公式

为了和 PyTorch `torch.optim.SGD(momentum=...)` 的默认思路对齐，本讲采用：

$$v_t = \mu v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

逐项解释：

- θ_t ：当前参数；

- $g_t = \nabla L(\theta_t)$: 当前 Step 的 Gradient;
- v_t : Momentum Buffer, 可理解为“带有历史记忆的梯度趋势”;
- μ : Momentum coefficient, 控制保留多少过去;
- η : Learning Rate, 最终把 buffer 转成参数更新步长。

如果:

$$\mu = 0$$

那么:

$$v_t = g_t$$

于是:

$$\theta_{t+1} = \theta_t - \eta g_t$$

就退化回普通 SGD。

第一条必须掌握的关系

Momentum 并没有取消当前 Gradient, 而是把“当前 Gradient + 一部分过去 Gradient 的历史”合成新的更新方向。

4 手算: Momentum 到底怎样记住过去?

取:

$$\mu = 0.9, \quad v_0 = 0$$

如果连续几个 Step 的 Gradient 都是:

$$g_1 = g_2 = g_3 = 1$$

那么:

$$v_1 = 0.9 \times 0 + 1 = 1$$

$$v_2 = 0.9 \times 1 + 1 = 1.9$$

$$v_3 = 0.9 \times 1.9 + 1 = 2.71$$

注意: 当前 Gradient 每次都只是 1, 但是 buffer 已经变成 2.71。

这表示:

如果很多步都在说“往同一个方向走”, Momentum 会越来越相信这个趋势。

反过来, 如果 Gradient 是:

$$1, -1, 1, -1, \dots$$

则:

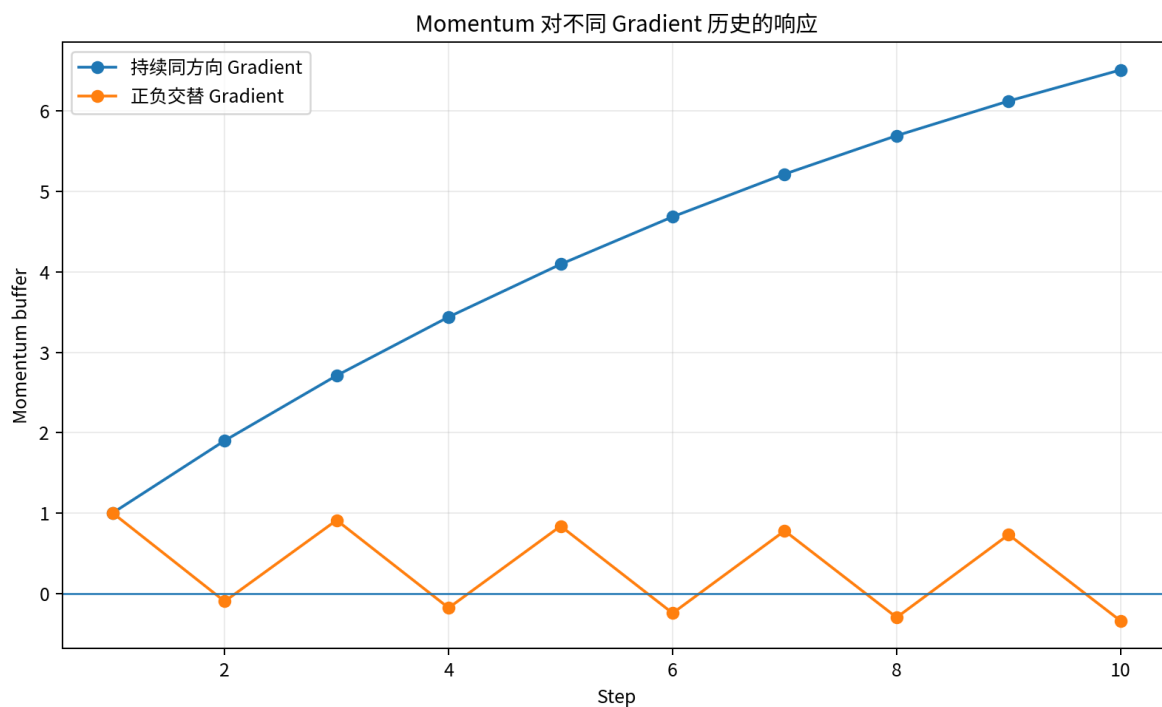
$$v_1 = 1$$

$$v_2 = 0.9 \times 1 - 1 = -0.1$$

$$v_3 = 0.9 \times (-0.1) + 1 = 0.91$$

$$v_4 = 0.9 \times 0.91 - 1 = -0.181$$

Buffer 的绝对值明显比“持续同方向”的情况小得多。



原理

持续方向被积累，频繁翻转方向被部分抵消。这就是 Momentum 能够在狭长山谷中减少横向震荡、同时沿稳定下降方向加快前进的核心直觉。

5 把公式展开：过去每一步到底占多大权重？

由：

$$v_t = \mu v_{t-1} + g_t$$

继续展开：

$$v_t = g_t + \mu g_{t-1} + \mu^2 g_{t-2} + \mu^3 g_{t-3} + \dots$$

所以过去 Gradient 的权重是：

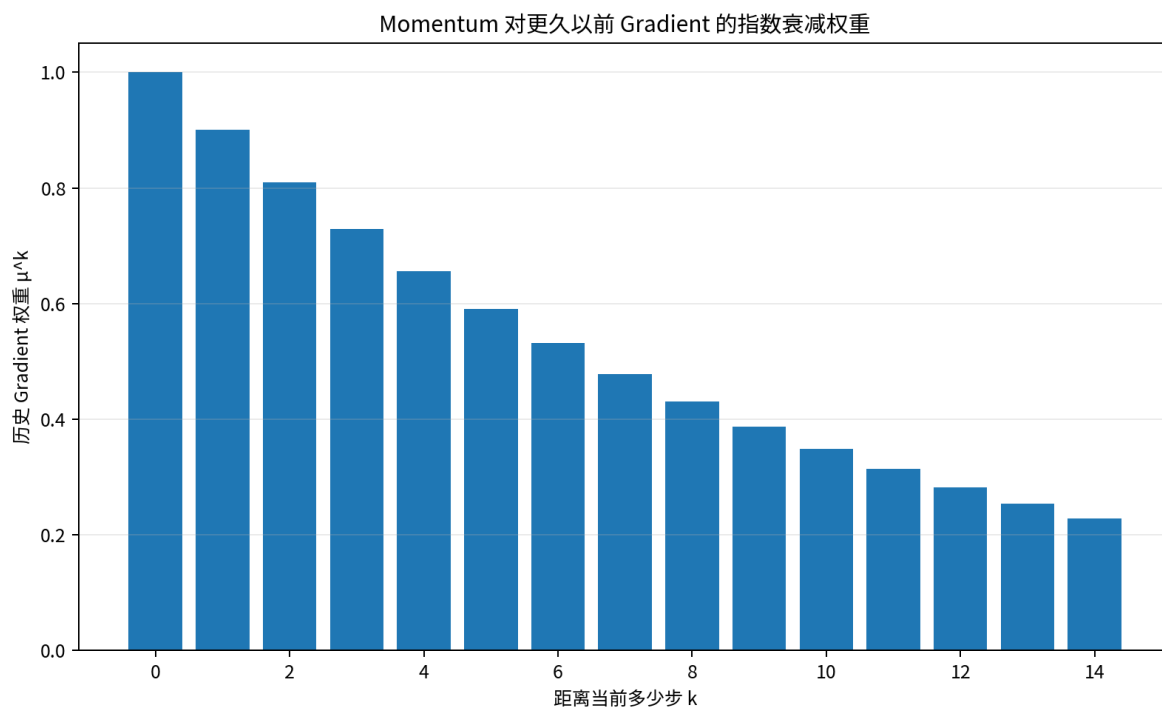
$$1, \mu, \mu^2, \mu^3, \dots$$

当：

$$\mu = 0.9$$

时：

$$1, 0.9, 0.81, 0.729, 0.6561, \dots$$



越久以前的 Gradient 权重越小，但不是突然消失，而是指数衰减。

例如 $\mu = 0.9$ 时，大约 6.58 步以前的 Gradient 权重下降到当前的 1/2 左右。常用的粗略“记忆长度”直觉是：

$$\frac{1}{1 - \mu}$$

因此 $\mu = 0.9$ 对应约 10 Step 的量级。但这只是直觉尺度，不是说第 11 步以前的信息突然变成 0。

6 为什么狭长 Loss Landscape 特别适合 Momentum?

考虑二维 Loss:

$$L(w_1, w_2) = \frac{1}{2}(w_1^2 + 50w_2^2)$$

Gradient:

$$\nabla L = \begin{bmatrix} w_1 \\ 50w_2 \end{bmatrix}$$

这里 w_2 方向曲率非常大， w_1 方向比较平缓。因此 Loss Landscape 像一条又长又窄的山谷。

如果 LR 取：

$$\eta = 0.04$$

普通 SGD 对 w_2 的更新是：

$$w_{2,t+1} = w_{2,t} - 0.04 \times 50w_{2,t} = -w_{2,t}$$

也就是说：

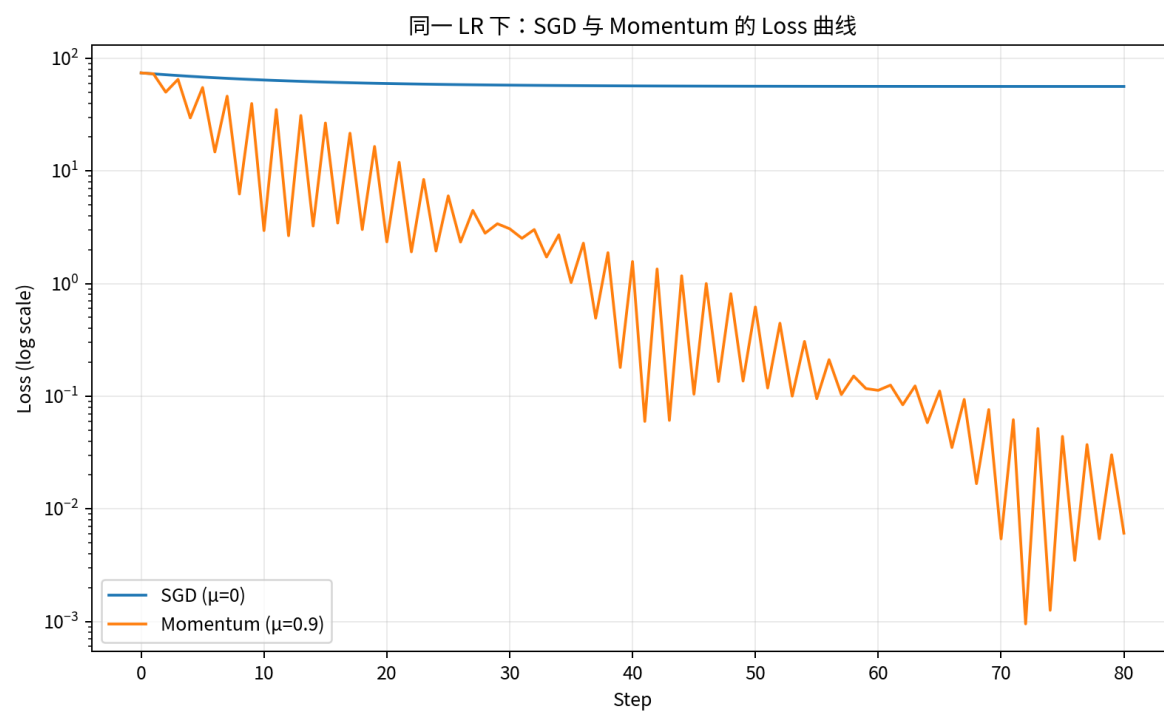
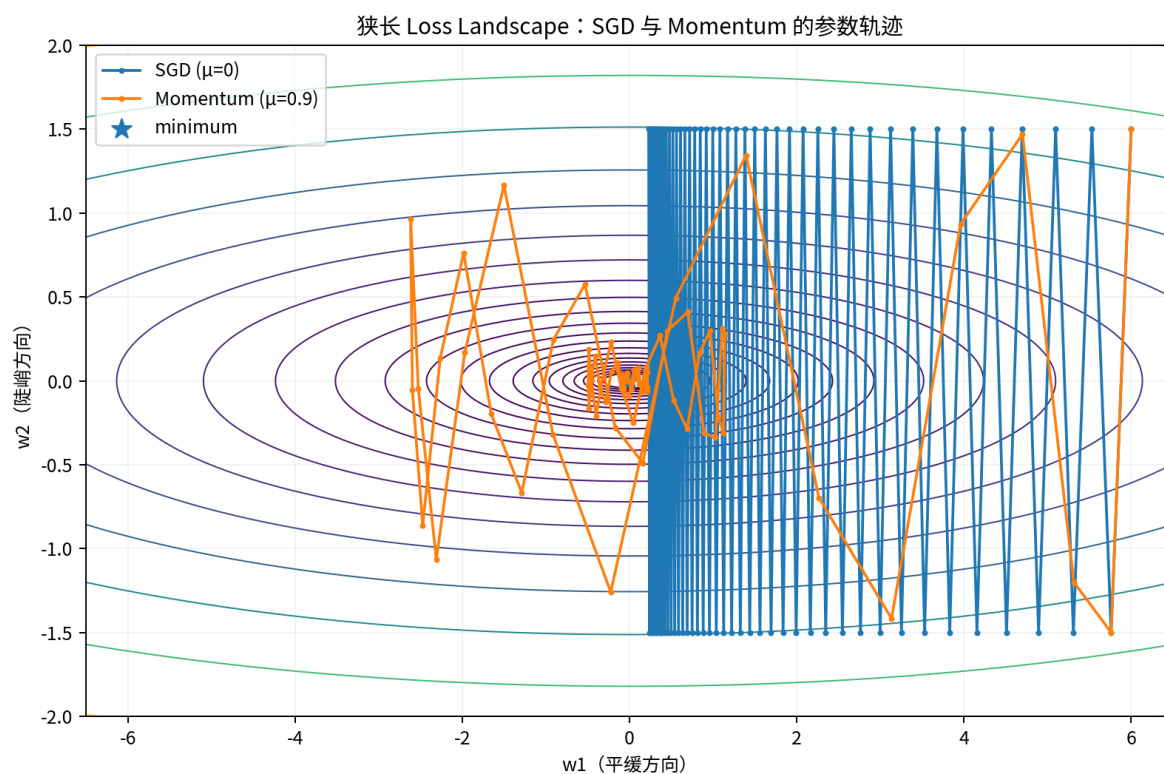
$$1.5 \rightarrow -1.5 \rightarrow 1.5 \rightarrow -1.5 \rightarrow \dots$$

它会永远在谷底两边等幅翻转。

而 w_1 方向：

$$w_{1,t+1} = 0.96w_{1,t}$$

虽然在慢慢靠近 0，但整个 Loss 仍然被 w_2 的持续震荡主导。



本讲实际实验：

- 起点 $[6, 1.5]$;

- $LR = 0.04$;
- 普通 SGD 80 Step 后 Loss 仍约 56.28;
- Momentum $\mu = 0.9$ 80 Step 后 Loss 约 0.0061。

为什么 Momentum 能减少震荡?

在陡峭方向, Gradient 正负不断翻转: 正、负、正、负。Momentum Buffer 会让这些相反历史贡献互相抵消, 因此不再每一步都全力横跳。

在平缓方向, Gradient 长期保持同一个方向。Momentum 会累积该方向, 使参数沿山谷长度方向推进得更快。

7 Momentum 不是简单的 “Gradient 平滑器”

有时会听到:

Momentum 就是在平滑 Gradient。

这个说法有一定直觉价值, 但不够完整。

更准确地说:

1. Momentum Buffer 是历史 Gradient 的加权和;
2. 它确实降低某些高频正负翻转的影响;
3. 但它同时会在持续方向上积累幅度;
4. Buffer 最终通过 Learning Rate 直接决定参数更新;
5. 因此它不仅 “平滑”, 还改变了优化动力学和参数轨迹。

8 μ 越大越好吗? 不一定

Momentum coefficient μ 控制历史保留程度。

如果:

$$\mu = 0$$

就是普通 SGD。

如果:

$$\mu \approx 0.9$$

过去信息保留较多, 是非常常见的设置。

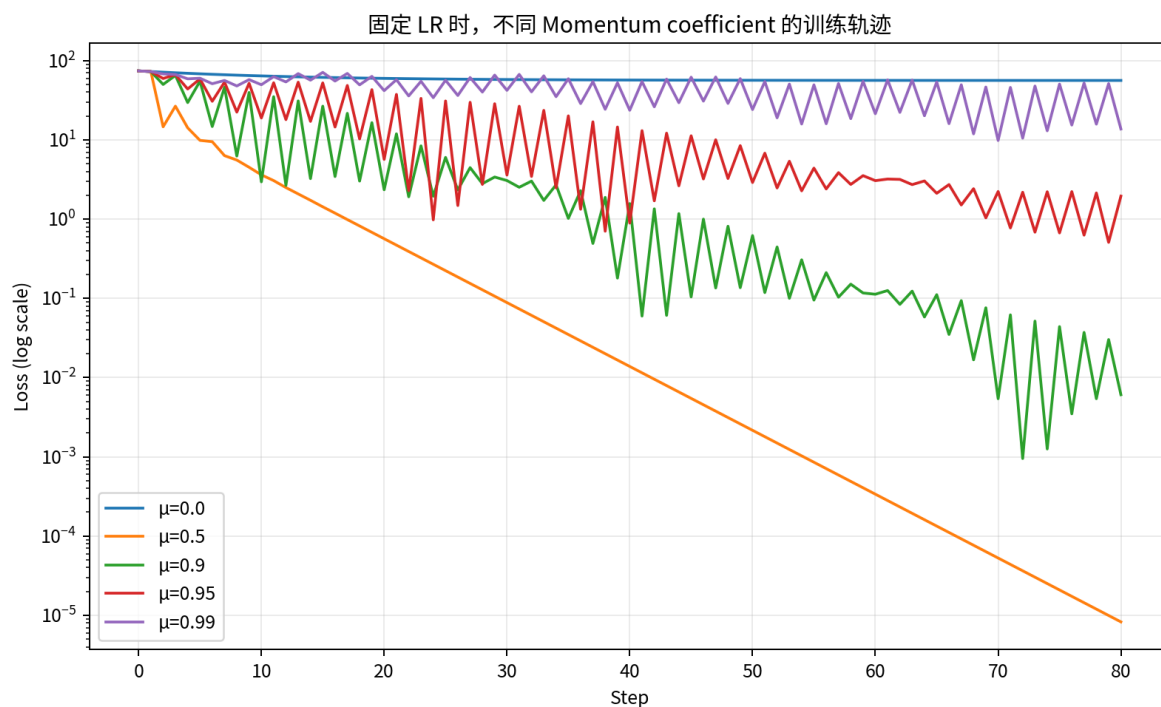
如果:

$$\mu \rightarrow 1$$

历史消退得非常慢, 惯性很强。

但 “记得更久” 不等于 “训练更好”。过强的惯性可能:

- 让参数冲过合适区域；
- 在 Loss Landscape 改变方向以后仍继续沿旧方向运动；
- 与较大的 LR 组合产生持续振荡；
- 对非平稳 Gradient 反应过慢。



在本讲固定 LR 的特定实验中：

μ	Step 80 左右 Loss	现象
0	≈ 56.28	普通 SGD 持续横向震荡
0.5	$\approx 8 \times 10^{-6}$	本例最快
0.9	≈ 0.0061	明显优于普通 SGD
0.95	≈ 1.95	惯性较强，收敛较慢
0.99	≈ 13.74	旧趋势保留过久

常见误区

不能从这个小实验得出“ $\mu = 0.5$ 比 0.9 普遍更好”。真正结论是： μ 与 LR、Loss 曲率、Batch Size、Gradient Noise、训练阶段共同决定结果。优化器超参数不能拆开机械比较。

9 Learning Rate 与 Momentum 是耦合的

普通 SGD：

$$\Delta\theta_t = -\eta g_t$$

Momentum：

$$\Delta\theta_t = -\eta v_t$$

而 v_t 可能远大于当前 g_t ，因为它积累了历史。

因此加入 Momentum 后，即使 nominal LR 不变，实际 Update Norm 也可能明显变化。

所以：

“我只把 momentum 从 0 改成 0.9，其他完全一样，因此这是一个纯单变量比较”

在数学上仍然是一个有效的 ablation，但要意识到：它不仅改变“记忆”，也改变了实际参数更新的动力学和尺度。

科研实验中的建议

如果论文比较 SGD 与 SGD+Momentum，至少记录：

- Learning Rate;
- Momentum coefficient;
- Batch Size / Effective Batch;
- LR Scheduler;
- Gradient clipping;
- 总 Optimizer Steps / Tokens Seen;
- 是否对两个优化器分别调过 LR。

否则“谁更好”的结论可能混入超参数适配程度差异。

10 为什么不同教材的 Momentum 公式长得不一样？

你以后可能看到：

$$v_t = \mu v_{t-1} - \eta g_t$$

$$\theta_{t+1} = \theta_t + v_t$$

也可能看到：

$$v_t = \mu v_{t-1} + (1 - \mu)g_t$$

还有本讲采用的：

$$v_t = \mu v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

这些公式的变量定义和缩放方式不同。关键是不要只看 v 的符号，而要看：

1. 历史项怎样衰减；
2. 当前 Gradient 乘了什么系数；
3. Learning Rate 是放在 Buffer 里面还是外面；
4. 最终 θ 到底怎样更新。

读优化器公式的方法

不要机械背一个“唯一正确的 Momentum 公式”。先找出：**状态变量是什么、历史怎样进入、当前 Gradient 怎样进入、最终参数更新是多少。**

11 PyTorch 中的 Momentum 到底发生了什么？

代码：

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.05,
    momentum=0.9
)
```

Optimizer 内部会为 Parameter 维护 momentum buffer。第一次遇到 Gradient 时, Buffer 从当前 Gradient 开始；后续 Step 继续：

$$v_t = \mu v_{t-1} + g_t$$

然后参数更新：

$$\theta \leftarrow \theta - \eta v_t$$

本讲实验使用二维 Parameter，连续 8 Step 手写 Momentum，再与 PyTorch 做逐 Step 对照，最大误差仅为 10^{-16} 量级。

这说明：

PyTorch 并没有使用神秘机制，它只是自动维护了我们手算过的 Optimizer State。

12 Optimizer State 是什么？

普通 SGD（无 Momentum）除了 Parameter 本身，通常不需要额外历史状态。

Momentum 需要为每个 Parameter 保存类似：

$$v_t$$

因此模型有 1 亿个 Parameter 时，Momentum Buffer 本身也可能需要接近 1 亿个对应数值。

这开始引出一个以后训练 LLM 很重要的概念：

Optimizer State 也占显存 / 内存。

当进一步进入 Adam 时，每个 Parameter 常常还需要一阶矩、二阶矩等状态，因此训练显存远不只是“模型参数大小”。本课程暂时只建立这个直觉，不展开分布式 Optimizer State。

13 Nesterov Momentum：现在理解到什么程度就够？

PyTorch 还支持：

```
nesterov=True
```

普通 Momentum 可以粗略理解为：

先根据历史和当前 Gradient 形成惯性，再更新。

Nesterov 的思想更像：

先考虑惯性可能把参数带到哪里，再在“前瞻位置”修正下降方向。

当前阶段你只需要知道它是 Momentum 的一个改进变体，不需要推完整公式。未来论文真正用到时再 Just-in-Time 深挖。

14 Momentum 与 Adam 的关系

Adam 中会维护 first moment：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

它与 Momentum 的共同点是：

都不只看当前 Gradient，而会保存 Gradient 历史。

区别是 Adam 还会维护 second moment：

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

并进行 bias correction，再做按坐标自适应缩放。

因此可以把本讲看成理解 Adam 的第一块地基：

SGD → Momentum → Adam first moment → Adam / AdamW

15 与 LLM / Transformer 的关系

现代 LLM 训练通常不直接使用最朴素的 SGD，但 Momentum 思想并没有消失。Adam / AdamW 中的 β_1 本质上就承担着类似“历史 Gradient 一阶趋势”的角色。

以后论文里看到：

```
AdamW
beta1 = 0.9
beta2 = 0.95
learning_rate = ...
warmup_steps = ...
```

你应该知道：

- β_1 控制一阶历史 Gradient 的记忆；
- β_2 控制平方 Gradient 的历史统计；
- LR 决定最终更新尺度；
- Warmup / Scheduler 又会让 LR 随 Step 改变；
- Batch Size 和 Gradient Noise 会改变输入给 Optimizer 的 g_t 。

所以这些不是互相独立的几个配置数字，而是一套优化动力学。

16 与 Agent Systems 科研的关系

你的主方向是 Agent Systems，很多实验并不需要训练基础模型，但仍可能涉及：

- LoRA / QLoRA；
- reward model / verifier；
- policy fine-tuning；
- 小型预测器、router、critic；
- 表征层、memory encoder；
- 对 Agent trace 的监督学习。

这些训练代码只要使用 Adam / AdamW，就已经在间接使用 Momentum-like 的历史一阶信息。

另外，Agent trajectory 往往相关性强、分布非平稳，Gradient 方向可能随阶段变化。此时“历史应该保留多久”就不只是抽象数学问题，而可能影响训练对新行为分布的响应速度。

读论文时可以问的三个问题

1. 优化器是否有 Momentum / β_1 ？数值是多少？
2. 训练数据分布是否随时间变化，历史 Gradient 是否可能滞后？
3. 如果做优化器 Ablation，是否同时公平调整 LR、Scheduler 和训练预算？

17 完整手算 5 Step：Gradient、Buffer、Parameter 如何一起变化？

很多初学者能看懂公式，但一进入代码就又把 g_t 、 v_t 、 θ_t 混在一起。下面用一个最小数字例子把三者彻底分开。

设：

$$\theta_0 = 10, \quad \eta = 0.1, \quad \mu = 0.9, \quad v_0 = 0$$

假设前 5 个 Step 的 Gradient 已经由 Backpropagation 算出：

$$g = [2, 2, 1, -1, 0.5]$$

第 1 步:

$$v_1 = 0.9 \times 0 + 2 = 2$$

$$\Delta\theta_1 = -0.1 \times 2 = -0.2$$

$$\theta_1 = 10 - 0.2 = 9.8$$

第 2 步:

$$v_2 = 0.9 \times 2 + 2 = 3.8$$

$$\Delta\theta_2 = -0.1 \times 3.8 = -0.38$$

$$\theta_2 = 9.42$$

注意: 第 2 步当前 Gradient 仍然只是 2, 但实际更新幅度从 0.2 增大到了 0.38, 因为历史方向被积累了。

第 3 步:

$$v_3 = 0.9 \times 3.8 + 1 = 4.42$$

$$\Delta\theta_3 = -0.442$$

$$\theta_3 = 8.978$$

第 4 步 Gradient 突然变成 -1:

$$v_4 = 0.9 \times 4.42 - 1 = 2.978$$

虽然当前 Gradient 已经要求“反方向”, 但 Buffer 仍然为正, 因为过去连续的正 Gradient 惯性还没有完全消失。

这一步极其重要:

Momentum 不会在 Gradient 第一次翻转时立刻把历史全部忘掉。

第 5 步:

$$v_5 = 0.9 \times 2.978 + 0.5 = 3.1802$$

因此 Momentum 的状态是一个真正跨 Step 存在的动态变量。训练过程不能只看某一时刻的 Gradient, 还要看此前积累的 Optimizer State。

为什么这会带来风险?

如果 Loss Landscape 的下降方向真的发生了长期改变, 而不是一次随机抖动, 过强的 Momentum 可能继续沿旧趋势前进若干步。因此 Momentum 既可以过滤短期噪声, 也可能造成“反应滞后”。这就是 μ 不能无限增大的原因之一。

18 Gradient Noise 与 Valley Oscillation: 两个问题不要混成一个

Momentum 常被用来解释两个现象: **Mini-Batch Gradient Noise** 和 **狭长山谷中的方向震荡**。它们有关联, 但不是同一个问题。

情形 A: Gradient Noise

同一个参数位置，如果换不同 Mini-Batch:

$$g_t = g^* + \epsilon_t$$

其中 g^* 可以理解为更稳定的总体趋势， ϵ_t 是抽样噪声。

如果噪声正负快速变化，历史加权可以让部分噪声互相抵消，因此更新方向更稳定。

情形 B: 高曲率方向震荡

即使没有随机 Batch Noise，在确定性的 Full-Batch 二次函数上，只要 LR 与曲率组合不合适，也会出现：

$$+, -, +, -, +, -$$

这样的 Gradient 翻转。

本讲二维二次 Loss 的 SGD 震荡就是完全确定性的，不是因为 Mini-Batch 随机性。

科研阅读时的区别

看到论文说 “Momentum reduces oscillation” 时，要问：作者指的是随机梯度噪声、Loss curvature 导致的确定性 zig-zag，还是两者都有？不同机制需要不同实验才能区分。

19 Momentum Buffer 是不是 Exponential Moving Average?

这里需要一个术语上的精确区分。

本讲 PyTorch-style 形式：

$$v_t = \mu v_{t-1} + g_t$$

展开后是历史 Gradient 的指数衰减加权，但权重总和并不被归一化到 1。

而标准指数移动平均常写成：

$$m_t = \mu m_{t-1} + (1 - \mu)g_t$$

这时历史权重之和趋近于 1，更像真正的加权平均。

两者具有相同的“指数衰减记忆”结构，但数值尺度不同。

例如恒定 Gradient：

$$g_t = 1$$

则 PyTorch-style buffer 最终趋近：

$$\frac{1}{1 - \mu}$$

当 $\mu = 0.9$ 时趋近 10。

而 EMA 形式会趋近 1。

原理

所以“Momentum 是 Gradient 的指数移动平均”作为直觉可以接受，但阅读公式和实现时必须检查是否存在 $(1 - \mu)$ 缩放。否则会误判 Update Magnitude。

20 PyTorch 代码逐行对应数学

最小训练代码：

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.05,
    momentum=0.9
)
```

```
optimizer.zero_grad()
pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```

逐行对应：

1. `zero_grad()`：清空 Parameter 上一次留下的 g_{t-1} ；注意它通常不会清掉 Optimizer 内部的 momentum buffer；
2. `model(x)`：用当前 θ_t 做 Forward；
3. `loss_fn(...)`：得到当前 L_t ；
4. `loss.backward()`：通过 Backpropagation 得到当前 g_t ；
5. `optimizer.step()`：Optimizer 读取 g_t 和内部保存的 v_{t-1} ，计算 v_t ，再更新 θ_{t+1} 。

这里最容易误解的是第一点：

常见误区

`optimizer.zero_grad()` 清的是 Parameter 的 Gradient，不是把 Momentum 的历史状态也全部清零。如果每个 Step 都把 momentum buffer 清零，那么 Momentum 就失去跨 Step 记忆，几乎退化成普通 SGD。

21 模型 state_dict 与 Optimizer state_dict 为什么要分开？

模型 Parameter 通常保存在：

```
model.state_dict()
```

但 Momentum Buffer 属于 Optimizer State，通常保存在：

`optimizer.state_dict()`

如果只保存模型 Parameter，不保存 Optimizer State，然后中断训练再恢复：

- Forward 结果可以恢复；
- 但 Momentum 历史会丢失；
- Scheduler Step 等训练状态也可能需要额外保存；
- 恢复后的早期几个 Step 可能与“连续训练”轨迹不同。

这就是为什么正式科研训练 checkpoint 往往不仅保存模型权重，还要保存 Optimizer、Scheduler、Global Step、随机状态等。

对实验可复现性的意义

如果论文声称“从 checkpoint 无缝恢复训练”，只加载 model weights 并不一定足够。对于带 Momentum / Adam 的训练，Optimizer State 是训练轨迹的一部分。

22 如何诊断 Momentum 可能有问题？

如果训练出现异常，可以观察：

- Training Loss 是否长期振荡；
- Gradient Norm 是否突然改变，而 Update Norm 仍因 Momentum 保持很大；
- Parameter Update 与当前 Gradient 是否明显不再同方向；
- 降低 μ 后是否更快适应新方向；
- 降低 LR 后是否减少 overshoot；
- 换 Batch Size 后 Gradient Noise 是否明显改变。

如果具备日志条件，还可以记录：

$$\|g_t\|, \quad \|v_t\|, \quad \|\Delta\theta_t\|, \quad \frac{g_t^T v_t}{\|g_t\| \|v_t\|}$$

最后一个量是当前 Gradient 与 Momentum Buffer 的夹角余弦：

- 接近 1：历史趋势与当前方向一致；
- 接近 0：二者近乎正交；
- 小于 0：历史惯性与当前 Gradient 出现明显冲突。

这类诊断在复杂训练失败时比只看 Loss 一条曲线更有信息量。

23 常见误区

误区 1: Momentum 就是让 Learning Rate 自动变大

不准确。Momentum 改变的是基于历史 Gradient 的 update direction / magnitude; LR 仍然是独立超参数。二者共同决定实际更新。

误区 2: μ 越大越稳定

不一定。过大的 μ 可能让旧方向影响保留太久, 出现 overshoot、振荡或反应迟缓。

误区 3: Momentum 一定比 SGD 好

不成立。它经常有效, 但效果依赖问题几何、LR、噪声和调参。优化器比较必须在合理超参数下进行。

误区 4: Momentum Buffer 就是模型 Parameter

不是。它属于 Optimizer State, 用于决定更新; 通常不参与模型 Forward, 也不是模型推理时必须保留的 Parameter。

误区 5: 训练 Loss 更快下降就说明泛化更好

仍然不成立。Momentum 可以改变 Optimization speed, 但 Test / Validation generalization 必须单独评估。

24 本讲总结: 最低闭环

你至少应该能够自己解释下面这条链:

$$g_t \rightarrow v_t = \mu v_{t-1} + g_t \rightarrow \Delta\theta = -\eta v_t \rightarrow \theta_{t+1}$$

并真正理解:

1. 普通 SGD 只看当前 Gradient;
2. Momentum 为 Optimizer 加入历史状态;
3. 持续同方向会被强化;
4. 正负交替方向会部分抵消;
5. 狭长山谷中, 这有助于减少横向震荡、加快纵向前进;
6. μ 与 LR 必须一起看;
7. PyTorch 会自动维护 momentum buffer;
8. Adam 的 first moment 延续了同样的“历史 Gradient”思想。

下一讲

第 38 讲进入 **Adam**。届时会在 Momentum 的“历史一阶方向”上，再加入“Gradient 大小的历史统计”，理解为什么 Adam 可以对不同参数方向使用不同的自适应缩放。